

Practice 06 — First Steps in AWS

Objective: set up a safe AWS working environment and drive it from the AWS CLI. That means a secured account, a budget alert, a CLI profile, and listing regions and AZs. These are the skills from today's lecture.

Timebox: ~60–75 min of actual work. If you're stuck on one step for more than 10 minutes, ask instead of pushing through alone.

AWS access isn't guaranteed for everyone, so there are two tracks. Do **Track A** if you have your own AWS account, or can create one. Otherwise do **Track B**, which needs no account and no card. Both tracks finish with the same **Part C** (concept tasks) and the same report. Write in your report which track you did.

Definition of done

- ☐ Track A **or** Track B done, every step shown in the report with the command you ran and its output (or a screenshot for console steps)
 - ☐ Part C done
 - ☐ Questions answered
 - ☐ Cleanup done and shown
 - ☐ Report (`REPORT.md` or PDF) submitted the way your instructor says. Submitting it is also your attendance record for this session.
-

Track A — real AWS account

Use region `eu-central-1` (**Frankfurt**) for everything, so that everyone's screenshots match.

A1. Secure the root user

1. Sign in to the console as the **root user** (the sign-up email).
2. Top-right menu → **Security credentials** → **Assign MFA device**. Use an authenticator app (Google Authenticator, Microsoft Authenticator, Authy, ...).
3. Screenshot: the MFA device listed for root. **Hide** the QR code and any account numbers you don't want to share.

Never create access keys for the root user.

A2. Create an admin identity for daily work

1. Open **IAM** → **User groups** → create the group `Admins` and attach the AWS managed policy `AdministratorAccess`.
2. **Users** → create the user `<yourname>-admin` with console access, and add it to `Admins`.
3. Sign out of root, then sign in as `<yourname>-admin` using the account's IAM sign-in URL (shown on the IAM dashboard).
4. Assign an MFA device to this user too.
5. Screenshot: the IAM user, its group, and its MFA device.

A3. Create a \$1 budget

1. **Billing and Cost Management** → **Budgets** → **Create budget**.
2. Type: cost budget, monthly, amount **\$1.00**.
3. Alert threshold: 80% of actual spend, sent to your email.
4. Screenshot: the budget in the list.

A budget only sends alerts. It does **not** stop anything.

A4. Configure the CLI

1. Install the **AWS CLI v2**: <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>
2. In IAM, open your admin user → **Security credentials** → **Create access key** → choose *Command Line Interface (CLI)*. Copy both values. The secret is shown only once.
3. Configure a named profile:

```
aws --version
aws configure --profile css4003
# Access key ID, secret, region: eu-central-1, output: json
aws sts get-caller-identity --profile css4003
```

The `Arn` in the output must end in `user/<yourname>-admin`. If it says `root`, you used the wrong credentials.

A5. Explore regions and AZs

```
export AWS_PROFILE=css4003
aws ec2 describe-regions --query 'Regions[].RegionName' --output text
aws ec2 describe-regions --all-regions \
  --query 'Regions[][RegionName,OptInStatus]' --output table
aws ec2 describe-availability-zones --region eu-central-1 \
  --query 'AvailabilityZones[][ZoneName,ZoneId,State]' --output table
```

Paste the outputs into your report.

A6. Cleanup

- **Deactivate and delete the access key** you made in A4 (IAM → your user → Security credentials), unless your instructor says to keep it for next week. Then run `aws sts get-caller-identity --profile css4003` again and paste the error, which proves the key is dead.
- Keep the MFA settings, the admin user, and the budget. They're meant to stay.
- You created nothing billable today, but open **Billing** → **Bills** anyway and screenshot the current month at \$0.00.

Track B — no AWS account

B1. Estimate a cost with the Pricing Calculator

The AWS Pricing Calculator needs no login: <https://calculator.aws>

1. **Create estimate** → set the location to **Europe (Frankfurt)**.
2. Add **Amazon EC2**: 1 Linux instance, `t3.micro`, on-demand, running **24 hours a day for a month**, with 8 GB of gp3 EBS storage.
3. Add **Amazon S3 Standard**: 5 GB stored.
4. Note the monthly total. Then change the EC2 usage to **8 hours a day, weekdays only** and note the new total.
5. Change the location to **Asia Pacific (Mumbai)** with the same settings and note the total.
6. Put the three totals in a table in your report. Use **Share** to get a link to the estimate and add the link too.

B2. Install the tools

You need Python 3 and the AWS CLI. **moto** is an open-source library that pretends to be AWS on your own machine. Nothing touches the real cloud and nothing costs money.

```
python3 -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install "moto[server]" awscli
aws --version
```

(If you already have the AWS CLI v2 installed, use it and skip `awscli` in the `pip install`.)

Start the mock in a **separate terminal** and leave it running:

```
source .venv/bin/activate
MOTO_IAM_LOAD_MANAGED_POLICIES=true moto_server -p 5000
# Windows PowerShell:
# $env:MOTO_IAM_LOAD_MANAGED_POLICIES="true"; moto_server -p 5000
```

`MOTO_IAM_LOAD_MANAGED_POLICIES=true` loads AWS's built-in policies, such as `AdministratorAccess`. Without it, step B5 fails with `NoSuchEntity`.

B3. Configure a CLI profile

moto accepts any credentials. Use fake ones:

```
aws configure set aws_access_key_id test --profile mock
aws configure set aws_secret_access_key test --profile mock
aws configure set region eu-central-1 --profile mock
aws configure set output json --profile mock
cat ~/.aws/config ~/.aws/credentials
```

Every command from here on needs both flags. Save typing with a variable:

```
E="--profile mock --endpoint-url http://localhost:5000"
aws sts get-caller-identity $E
```

(In zsh, which is the default on macOS, write `aws sts get-caller-identity ${=E}` instead, or just type the flags out each time.)

B4. Explore regions and AZs

```
aws ec2 describe-regions $E --query 'Regions[].RegionName' --output text
aws ec2 describe-regions --region-names eu-central-1 ap-south-1 me-central-1 \
  $E --output table
aws ec2 describe-availability-zones $E \
  --query 'AvailabilityZones[].ZoneName' --output text
```

Look at the `OptInStatus` column in the table output.

B5. Do the account setup from Track A, via the CLI

```
# admin group with AdministratorAccess
aws iam create-group --group-name Admins $E
aws iam attach-group-policy --group-name Admins \
  --policy-arn arn:aws:iam::aws:policy/AdministratorAccess $E
aws iam list-attached-group-policies --group-name Admins $E

# your admin user, in the group, with an access key
aws iam create-user --user-name yourname-admin $E
aws iam add-user-to-group --user-name yourname-admin --group-name Admins $E
aws iam list-groups-for-user --user-name yourname-admin $E
aws iam create-access-key --user-name yourname-admin $E

# a $1 monthly budget (123456789012 is moto's fake account ID)
aws budgets create-budget --account-id 123456789012 --budget \
  '{"BudgetName":"one-dollar","BudgetLimit":
{"Amount":"1","Unit":"USD"},"TimeUnit":"MONTHLY","BudgetType":"COST"}' $E
aws budgets describe-budgets --account-id 123456789012 $E
```

moto mimics the AWS **API**, not all of AWS's behaviour. It won't send budget emails or check MFA. Treat it as practice for the commands.

B6. Cleanup

Delete everything in reverse order, and paste the outputs:

```
aws iam list-access-keys --user-name yourname-admin $E
aws iam delete-access-key --user-name yourname-admin --access-key-id <KEY_ID> $E
aws iam remove-user-from-group --user-name yourname-admin --group-name Admins $E
aws iam delete-user --user-name yourname-admin $E
aws iam detach-group-policy --group-name Admins \
  --policy-arn arn:aws:iam::aws:policy/AdministratorAccess $E
aws iam delete-group --group-name Admins $E
aws budgets delete-budget --account-id 123456789012 --budget-name one-dollar $E
```

Then stop `moto_server` with `Ctrl+C`. It keeps everything in memory, so stopping it wipes all state. Run `aws configure` without `--profile mock` later and you'll see that your default profile was never touched.

Part C — concept tasks (both tracks)

C1. Classify each one as IaaS, PaaS, or SaaS. Give one line of reasoning for each.

Service	Model	Why
Amazon EC2		
Gmail		
AWS Elastic Beanstalk		
GitHub		
Amazon RDS		
A VM you rent from a hosting company and SSH into		

C2. Shared responsibility. For each task, write **AWS** or **Customer**.

Task	EC2	RDS	S3
Patching the operating system			
Physical security of the data center			
Deciding who can access the data			
Firewall / network access rules			
Replacing a failed disk			
Turning on encryption / backups settings			

Questions (answer in the report)

1. Why shouldn't you use the root user day to day? Name two tasks that genuinely need root.
2. What's the difference between a region and an Availability Zone? Why would you spread an app across two AZs?
3. Which AWS region would you pick for an app whose users are in Almaty, and why? Name at least two of the four factors from the lecture.
4. Your budget is \$1 and you've spent \$3. What did AWS do about it, and what do you have to do?
5. Name two things that keep costing money after you **stop** an EC2 instance or "stop using" an account.
6. You find an access key in a public GitHub repo. In what order do you (a) remove it from the repo, (b) deactivate it in IAM, (c) check what it was used for? Why that order?

Common mistakes

- `get-caller-identity` shows `root`. You created access keys for root, or configured the CLI with them. Delete those keys and use your IAM admin user's keys instead.
- "I can't see the thing I created." The console's region selector (top-right) is set to a different region. IAM is global, but almost everything else is regional.

- **Could not connect to the endpoint URL (Track B).** `moto_server` isn't running, or it's on a different port from the one in `$E`.
- **NoSuchEntity ... AdministratorAccess (Track B).** You started moto without `MOTO_IAM_LOAD_MANAGED_POLICIES=true`. Restart it with the variable set and redo B5. Restarting wipes moto's state.
- **The zsh variable doesn't expand (Track B).** In zsh, `aws ... $E` passes the whole string as one argument. Use `${=E}` or bash.
- **Screenshots leak secrets.** Never put a secret access key, an MFA QR code, or your full account email in the report. Blur them.